

Decomposing Algebraic Roots

Into Sums and Products of Minimum Degree

Exact Mathematica algorithms, optimality certificates,
and a complete finite method for the additive problem

September 6, 2026

Abstract

Given a specified algebraic number, how can one express it as a sum or a product of algebraic numbers while minimizing the largest absolute degree of a component? This is not ordinary polynomial factorization, and finding a plausible decomposition is not the same as proving its optimality. We develop exact resultant constructions, rational-coefficient discovery methods, finite exhaustive searches, root-selection certificates, and lower bounds that apply to arbitrarily many components. Both degree-nine examples in the motivating question have globally optimal maximum component degree three.

For sums, a normalized-trace argument gives a stronger general result: every decomposition can be replaced, without increasing any component degree, by one lying in the normal closure of the target field. Consequently, the unrestricted additive minimum is computable by a finite collection of rational fixed-space calculations. A reference implementation of this algorithm is supplied. For products, we explain precisely why the analogous norm argument does not give the same conclusion, and provide certified bounded searches without advertising them as an unrestricted terminating optimizer.

The accompanying archive contains Wolfram Language source, examples, regression specifications, and an independently executed SymPy verification script. The distinction between mathematical proofs, independently checked computations, and Wolfram code not executed in the authoring session is explicit.

Main result for the two supplied examples: maximum degree 3, with exact identities and a proof that no number of rational or quadratic components can do better.

Contents

1	The problem that is actually being optimized	2
2	The answers to the two examples	2
3	Forward construction with resultants	3
3.1	A minimal polynomial divides the resultant; it need not equal it	4
3.2	Why companion matrices give the same construction	4
4	A small exact implementation	4
4.1	Using the supplied package	5
5	Discovering candidate polynomials by coefficient equations	5
5.1	A particularly small ansatz for the supplied examples	6
5.2	Which normalizations are legitimate?	6
6	Finite exhaustive discovery, including several components	7
6.1	What a negative result does and does not prove	8
7	Lower bounds that apply to arbitrarily many components	8
8	A complete algebraic proof of the supplied identities	9
9	The unrestricted sum problem has a finite reduction	10
9.1	Constructing the fixed spaces exactly	11
9.2	A certificate of nonmembership	11
9.3	What the implementation actually constructs	11
9.4	A real-summand variant	12
10	Why searching only the target field is not enough	12
10.1	More than two components can be essential	13
11	The product problem: what norms do and do not settle	13
12	Implementation contract and practical workflow	14
12.1	Exiting nested searches reliably	15
13	Validation, reproducibility, and limitations	15
14	Conclusion	15
A	Archive contents and build instructions	16

1 The problem that is actually being optimized

The motivating Mathematica Stack Exchange question asks for inverse operations to the familiar conversion of sums and products of algebraic numbers into a single `Root`. The posted approaches use companion matrices and resultants with prescribed factor shapes [1]. Those are useful discovery tools. Here we distinguish their guarantees from the unrestricted minimization problem.

Fix an embedding $\overline{\mathbb{Q}} \subset \mathbb{C}$. The degree of an algebraic number always means

$$\deg(\beta) = [\mathbb{Q}(\beta) : \mathbb{Q}],$$

not the degree of a polynomial over an unspecified coefficient field.

Definition 1.1. For $\alpha \in \overline{\mathbb{Q}}$, define

$$D_+(\alpha) = \min \left\{ \max_i \deg(\beta_i) : r \geq 1, \quad \alpha = \sum_{i=1}^r \beta_i \right\}.$$

For $\alpha \neq 0$, define

$$D_\times(\alpha) = \min \left\{ \max_i \deg(\beta_i) : r \geq 1, \quad \alpha = \prod_{i=1}^r \beta_i, \quad \beta_i \neq 0 \right\}.$$

We set $D_\times(0) = 1$, using the one-factor representation 0.

Both minima exist: the one-component representation gives an upper bound $\deg(\alpha)$, and the possible costs are positive integers. A rational number has cost one. Requiring at least two components changes nothing, because one may append 0 to a sum or 1 to a product.

Rational coefficients in a decomposition need not be treated separately: $c\beta$ has degree at most $\deg(\beta)$ for $c \in \mathbb{Q}$. Similarly, a nonzero algebraic inverse has the same degree as the original number, so quotients may be represented as products. The objective is *not* to minimize the number of components, the coefficient heights, or the displayed expression length.

The coefficient-field convention is essential. Were algebraic coefficients allowed for free, the linear polynomial $X - \alpha$ would make the degree criterion vacuous. A nested equation such as $X^2 - \sqrt{2} = 0$ is quadratic over $\mathbb{Q}(\sqrt{2})$ but its positive root has absolute degree four. Thus radical denesting and the present optimization are related but different problems.

2 The answers to the two examples

Put

$$\begin{aligned} f(X) &= X^3 + X + 1, & g(X) &= X^3 - X + 1, \\ a &= \text{Root}(f, 1), & b &= \text{Root}(g, 1). \end{aligned}$$

Both cubics have exactly one real root. Numerically,

$$\begin{aligned} a &= -0.682327803828019327369483739711\dots, \\ b &= -1.32471795724474602596090885448\dots \end{aligned}$$

The two polynomials from the question are

$$P_\times(Z) = Z^9 + 2Z^7 - 3Z^6 + Z^5 - Z^4 + 3Z^3 - Z - 1, \tag{1}$$

$$P_+(Z) = Z^9 + 6Z^6 + 3Z^5 - 15Z^3 + 24Z^2 - 4Z + 8. \tag{2}$$

The exact identities are

$$\text{Root}(P_{\times}, 1) = ab, \quad ab = 0.903891894458347561098959026167\dots, \quad (3)$$

$$\text{Root}(P_{+}, 1) = a + b, \quad a + b = -2.00704576107276535333039259419\dots \quad (4)$$

Section 8 proves these identities, the degree-nine claims, and the selected branches without relying on numerical agreement.

The maximum degree three is globally optimal in each case. Any finite collection of quadratic extensions is contained in a multiquadratic extension of degree a power of two. Every element of that extension has degree dividing that power of two. A degree-nine number therefore cannot be a sum or a product of any number of algebraic numbers of degree at most two. The displayed cubic components attain the lower bound.

Here is a direct exact check in Mathematica; `RootReduce` is used as an equality verifier, not as a decomposition algorithm [3].

```
a = Root[1 + # + #^3 &, 1];
b = Root[1 - # + #^3 &, 1];

rTimes = Root[
  -1 - # + 3 #^3 - #^4 + #^5 - 3 #^6 + 2 #^7 + #^9 &, 1];
rSum = Root[
  8 - 4 # + 24 #^2 - 15 #^3 + 3 #^5 + 6 #^6 + #^9 &, 1];

{RootReduce[rTimes - a b], RootReduce[rSum - a - b]}
(* Expected: {0, 0} *)
```

The comments marked “Expected” are regression expectations. They are not claimed to be output from a Wolfram kernel in this authoring session; see Section 13.

3 Forward construction with resultants

Let $f, g \in \mathbb{Q}[X]$ be monic of degrees m, n , with roots $a_1, \dots, a_m$ and $b_1, \dots, b_n$, counted with multiplicity. Define

$$R_{+}(Z) = \text{Res}_X(f(X), g(Z - X)), \quad (5)$$

$$R_{\times}(Z) = \text{Res}_X(f(X), X^n g(Z/X)). \quad (6)$$

The second argument in (6) is a polynomial. If $g(X) = \sum_{j=0}^n g_j X^j$, then

$$X^n g(Z/X) = \sum_{j=0}^n g_j Z^j X^{n-j}.$$

Writing it this way avoids artificial divisions by zero or uncanceled negative powers.

Proposition 3.1 (Composed-polynomial identities). *The polynomials in (5)–(6) satisfy*

$$R_{+}(Z) = \prod_{i=1}^m \prod_{j=1}^n (Z - a_i - b_j),$$

$$R_{\times}(Z) = \prod_{i=1}^m \prod_{j=1}^n (Z - a_i b_j).$$

In particular, they are monic polynomials of degree mn over $\mathbb{Q}$.

Proof. For monic f , the defining evaluation identity for the resultant is $\text{Res}_X(f, h) = \prod_i h(a_i)$. Applying it to $g(Z - X)$ gives the first formula. The polynomial $X^n g(Z/X)$ is identically $\prod_j (Z - Xb_j)$; evaluating at each a_i gives the second. This identity remains valid when some $a_i = 0$. $\square$

The Wolfram implementation is `Resultant[poly1, poly2, x]` [4]. The proof also explains why no extra sign correction is needed after the input polynomials have been made monic.

3.1 A minimal polynomial divides the resultant; it need not equal it

If P is the minimal polynomial of a specified α , then

$$\alpha = a_i + b_j \implies P \mid R_+, \quad \alpha = a_i b_j \implies P \mid R_\times.$$

Conversely, if $P \mid R$ and α is any root of P , Proposition 3.1 guarantees that some root pair represents α . It does not identify the pair.

Equality $P = R$ follows when both are monic and $\deg P = mn$. Without that degree equality, coefficient matching against P can reject valid decompositions. For example, with $f = g = X^2 - 2$,

$$R_+(Z) = Z^2(Z^2 - 8), \quad R_\times(Z) = (Z^2 - 4)^2.$$

The particular sum $\sqrt{2} + \sqrt{2}$ has minimal polynomial $Z^2 - 8$, and the particular product is the rational number 2. Repeated resultant factors and degree drops are ordinary consequences of dependence among components.

3.2 Why companion matrices give the same construction

Let C_f, C_g be companion matrices. Their eigenvalues are the roots of f, g , so

$$\begin{aligned} \det(ZI - C_f \otimes C_g) &= R_\times(Z), \\ \det(ZI - (C_f \otimes I_n + I_m \otimes C_g)) &= R_+(Z). \end{aligned}$$

This follows either by diagonalization over a splitting field for separable inputs, or by the corresponding determinant identity. The resultant method avoids explicitly constructing an mn -dimensional matrix. Neither formulation by itself solves the inverse minimization problem.

4 A small exact implementation

The following pedagogical definitions assume monic rational input polynomials and distinct unsigned variables $\mathbf{x}$ and $\mathbf{z}$. The archive's package adds input checking, normalization, certificates, and failure handling.

```
ClearAll[sumPolynomial, productPolynomial, polynomialRoots,
  rootPair];

sumPolynomial[f_, g_, x_Symbol, z_Symbol] :=
  Expand[Resultant[f, g /. x -> z - x, x]];

productPolynomial[f_, g_, x_Symbol, z_Symbol] :=
  Module[{n = Exponent[g, x], c = CoefficientList[g, x]},
    Expand[Resultant[f,
      Sum[c[[j + 1]] z^j x^(n - j), {j, 0, n}], x]]
  ];
```

```

polynomialRoots[p_, x_Symbol] :=
  Table[Root[Function[Evaluate[p /. x -> Slot[1]]], k],
    {k, 1, Exponent[p, x]};

rootPair[alpha_, {f_, g_}, x_Symbol, op_] :=
  SelectFirst[
    Tuples[{polynomialRoots[f, x], polynomialRoots[g, x]}],
    TrueQ[RootReduce[Apply[op, #] - alpha] === 0] &,
    Missing["NoRootPair"]
  ];

```

Applied to the two cubics:

```

Clear[x, z];
f = x^3 + x + 1; g = x^3 - x + 1;

productPolynomial[f, g, x, z]
sumPolynomial[f, g, x, z]

rootPair[rTimes, {f, g}, x, Times]
rootPair[rSum, {f, g}, x, Plus]
(* In each case the expected pair is {a, b}. *)

```

The pair-selection step is essential. `Root[f, 1]` is not a universal way to choose a compatible conjugate. Mathematica orders real roots first, increasingly; nonreal-root ordering also involves the isolation representation [2]. Enumerating roots and proving equality avoids depending on a numerical ordering heuristic.

4.1 Using the supplied package

Load the file from the archive and request a certificate:

```

Get["RootDecomposition.wl"];

rp = RootPairFromPolynomials[rTimes, {f, g}, x, "Product"];
rs = RootPairFromPolynomials[rSum, {f, g}, x, "Sum"];

KeyTake[rp, {"Components", "Degrees", "MaximumDegree",
  "ElementaryLowerBound", "GloballyOptimal", "ExactResidual"}]

```

For the specified pair, the expected degrees are $\{3, 3\}$, the maximum and elementary lower bound are both 3, the exact residual is 0, and "GloballyOptimal" is `True`. The certificate stores the components separately, so ordinary evaluation or display simplification does not obscure what was measured.

Always compute degrees through `MinimalPolynomial` [5]. Inspecting the displayed polynomial or using `alpha[[1]]` indiscriminately is fragile: a rational or quadratic `Root` may already have simplified to another expression type.

5 Discovering candidate polynomials by coefficient equations

For a proposed degree pair (m, n) , write generic monic polynomials

$$f(X) = X^m + \sum_{i=0}^{m-1} u_i X^i, \quad g(X) = X^n + \sum_{j=0}^{n-1} v_j X^j.$$

Construct R_+ or $R_\times$. A necessary and sufficient condition for *these rational polynomials* to contain a representing root pair is

$$\text{rem}_Z(R(Z), P(Z)) = 0. \quad (7)$$

Equating the coefficients of this remainder gives polynomial equations in the u_i, v_j . If $mn = \deg P$, these are equivalent to equality of the monic polynomials.

The package builds the equations without silently imposing the wrong degree relation:

```
problem = PairCoefficientEquations[rTimes, {3, 3}, "Product"];
solutions = Solve[problem["Equations"], problem["Unknowns"]];
```

This is a candidate-generation step. The required coefficient domain is *rational*. An algebraic solution of the equations need not define factors of the advertised absolute degrees. Any free parameters must also be assigned rational values before accepting a candidate. After substitution, check all polynomial coefficients for rationality and call `RootPairFromPolynomials`.

5.1 A particularly small ansatz for the supplied examples

Taking

$$f_u(X) = X^3 + uX + 1, \quad g_v(X) = X^3 + vX + 1$$

gives, in the product case,

$$\begin{aligned} R_\times(Z) = & Z^9 - 2uvZ^7 - 3Z^6 + u^2v^2Z^5 + uvZ^4 \\ & + (u^3 + v^3 + 3)Z^3 + uvZ - 1. \end{aligned} \quad (8)$$

Matching (1) implies $uv = -1$ and $u^3 + v^3 = 0$. Over $\mathbb{Q}$ the solutions are $(u, v) = (1, -1)$ and $(-1, 1)$.

For sums, the Z^7 coefficient is $3(u + v)$, and the Z^5 coefficient is $3(u^2 + uv + v^2)$. Matching (2) gives $v = -u$ and $u^2 = 1$; substitution into the remaining coefficients confirms both possibilities.

```
Clear[u, v, x, z];
fu = x^3 + u x + 1; gv = x^3 + v x + 1;

rt = ComposedPolynomial[fu, gv, x, z, "Product"];
pt = MinimalPolynomial[rTimes, z];
Solve[Thread[CoefficientList[rt - pt, z] == 0], {u, v}]

rsym = ComposedPolynomial[fu, gv, x, z, "Sum"];
ps = MinimalPolynomial[rSum, z];
Solve[Thread[CoefficientList[rsym - ps, z] == 0], {u, v}]
```

The product system also has nonrational complex solutions. Keeping them without checking the coefficient field would confuse a cubic over an extension with a cubic over $\mathbb{Q}$.

5.2 Which normalizations are legitimate?

For sums, the transformation

$$(\beta, \gamma) \mapsto (\beta + t, \gamma - t), \quad t \in \mathbb{Q},$$

preserves rational coefficient fields. It can eliminate the next-to-leading coefficient of one candidate polynomial. Additional eliminations need additional justification.

For products, the analogous freedom is

$$(\beta, \gamma) \mapsto (t\beta, t^{-1}\gamma), \quad t \in \mathbb{Q}^\times.$$

If the first monic polynomial has degree m and constant coefficient c , scaling its root by t changes that coefficient to $t^m c$. Making it equal to one requires a rational solution of $t^m = 1/c$; this need not exist. Consequently, setting constant terms to one or assuming both polynomials are depressed trinomials is a useful *ansatz*, not a universal rational normalization.

6 Finite exhaustive discovery, including several components

Coefficient equations may have complicated rational solution sets. A slower but transparent alternative is to enumerate minimal polynomials within explicit bounds.

Define the naive height of a primitive integer polynomial by

$$H(c_0 + \cdots + c_m X^m) = \max_i |c_i|, \quad c_m > 0, \quad \gcd(c_0, \dots, c_m) = 1.$$

For fixed d, h , form the finite catalog of all irreducible such polynomials with degree at most d and height at most h . `IrreduciblePolynomialQ` supplies the exact irreducibility test [6]. *Nonmonic polynomials must be included*: otherwise the search would cover only algebraic integers and miss, for example, roots of $2X^2 - 1$.

For pairs, enumerate unordered polynomial pairs with repetition. Reject those with degree product smaller than $\deg \alpha$. For each remaining pair, compute its composed polynomial, test divisibility by the target minimal polynomial, and select a root pair exactly. Iterate the degree threshold in increasing order.

```
BoundedPairDecompose[rTimes, "Product", 3, 1]
BoundedPairDecompose[rSum, "Sum", 3, 1]
```

These bounds already contain the two cubics from the question. The implementation may discover an equivalent sign variant first; both the identity and its degrees are rechecked. Since the found maximum is three and the global lower bound is three, this bounded discovery supplies a *global* optimality certificate for these inputs.

For up to $r_{\max}$ components, compose the polynomials repeatedly, then enumerate compatible root tuples. The companion routine is

```
BoundedDecompose[alpha, "Product", d, h, rmax]
BoundedDecompose[alpha, "Sum", d, h, rmax]
```

It is deliberately a small-bound reference algorithm. It uses nondecreasing polynomial-index tuples to avoid testing permutations of the same tuple, while allowing repetitions.

Proposition 6.1 (Completeness within stated bounds). *Suppose α has a representation of the requested type with at most $r_{\max}$ components, each of degree at most d , and each primitive minimal polynomial of height at most h . Then the finite exhaustive search with those bounds finds a representation, assuming the exact arithmetic operations complete successfully.*

Proof. Every component's minimal polynomial belongs to the catalog. Its unordered polynomial tuple is therefore visited. Repeated application of Proposition 3.1 produces a polynomial vanishing at α , so the irreducible minimal polynomial of α divides it. The subsequent root-tuple enumeration includes the given components and accepts them by exact equality. $\square$

6.1 What a negative result does and does not prove

A return value "NotFoundWithinBounds" means only that the stated finite search was exhausted. It does not rule out greater height, more components, or a smaller-degree representation outside the selected family. The package explicitly records that global impossibility has not been proved. Increasing h and $r_{\max}$ gives a semidecision procedure for a fixed degree bound: it eventually finds a representation whenever one exists. It need not terminate when no representation exists. Searching forever at degree two before trying degree three is therefore not an optimizer. Dovetailing all positive searches avoids that starvation but still does not certify that an unobserved smaller-degree decomposition is impossible.

The raw catalog size in degree m is at most $h(2h+1)^m$ before primitive and irreducibility filtering. Tuple enumeration can grow as a power of the catalog size, and exact resultants can have large coefficients. Modular rejection filters, cached resultants, numerical prioritization followed by exact checks, and meet-in-the-middle searches can improve performance; none changes the mathematical meaning of the bounds.

7 Lower bounds that apply to arbitrarily many components

A bound such as $\deg \alpha \leq d^r$ is useful when r is fixed, but says little when arbitrary finite r is allowed. Splitting fields give stronger obstructions. We use the standard Galois correspondence and orbit interpretation of algebraic degree [10].

Theorem 7.1 (Largest-prime lower bound). *Let $n = \deg \alpha > 1$, and let $p_{\max}(n)$ be its largest prime divisor. Then*

$$D_+(\alpha) \geq p_{\max}(n), \quad D_\times(\alpha) \geq p_{\max}(n).$$

The same obstruction applies to any rational expression in algebraic inputs of uniformly bounded absolute degree.

Proof. Suppose all inputs β_i have degrees $m_i \leq d$. Let L_i be the splitting field of the minimal polynomial of β_i , and let E be their compositum. Restriction embeds $\text{Gal}(E/\mathbb{Q})$ into $\prod_i \text{Gal}(L_i/\mathbb{Q})$. Each factor embeds into S_{m_i} , so $[E : \mathbb{Q}]$ divides $\prod_i m_i!$. Every prime divisor of $[E : \mathbb{Q}]$ is therefore at most d .

The represented number belongs to E , hence $n = [\mathbb{Q}(\alpha) : \mathbb{Q}]$ divides $[E : \mathbb{Q}]$ by the tower law. Every prime divisor of n must be at most d . $\square$

For prime n , this proves $D_+(\alpha) = D_\times(\alpha) = n$ immediately. For $n = 9$, it gives three. The implementation is simply

```
DegreeLowerBound[alpha]
```

Do not replace the factorial argument by the generally false claim that $[\mathbb{Q}(\beta_1, \dots, \beta_r) : \mathbb{Q}]$ divides $\prod_i \deg \beta_i$. Distinct conjugate cubic fields can have compositum degree six rather than a divisor of nine.

Theorem 7.2 (Galois-exponent obstruction). *Let L be the normal closure of $\mathbb{Q}(\alpha)$ and $G = \text{Gal}(L/\mathbb{Q})$. If α is expressible using algebraic inputs of degree at most d and rational operations, then*

$$\exp(G) \mid \text{lcm}(1, 2, \dots, d).$$

Proof. With E as above, $\text{Gal}(E/\mathbb{Q})$ is a subgroup of a product of groups S_{m_i} . Their exponents divide $\text{lcm}(1, \dots, d)$, so the exponent of the subgroup does also. Since E is normal and contains

α , it contains L . Restriction maps $\text{Gal}(E/\mathbb{Q})$ onto G , and an exponent bound passes to a quotient. $\square$

Example 7.3 (Nested square roots need not have low absolute-degree components). Let $\alpha = \sqrt{2 + \sqrt{2}}$. Its minimal polynomial is

$$X^4 - 4X^2 + 2.$$

The other positive root is $\beta = \sqrt{2 - \sqrt{2}} = \alpha^3 - 3\alpha$, so the polynomial splits in $\mathbb{Q}(\alpha)$. The automorphism $\alpha \mapsto \beta$ sends β to $-\alpha$, and has order four. Thus the Galois group is cyclic of order four. Since $4 \nmid \text{lcm}(1, 2, 3) = 6$, no combination of inputs of degrees at most three can represent α using rational operations. In particular,

$$D_+(\alpha) = D_\times(\alpha) = 4.$$

Its nested quadratic description does not contradict this statement: the outer square root is taken over a nonrational coefficient field.

8 A complete algebraic proof of the supplied identities

The resultant computations from Section 3 give exactly

$$\text{Res}_X(f(X), g(Z - X)) = P_+(Z), \quad \text{Res}_X(f(X), Z^3 - ZX^2 + X^3) = P_\times(Z).$$

We now justify irreducibility and the selected real roots.

The cubics f and g have no rational roots, so they are irreducible. Their discriminants are -31 and -23 , respectively. Each has Galois group S_3 , and its splitting field has a unique quadratic subfield, respectively $\mathbb{Q}(\sqrt{-31})$ and $\mathbb{Q}(\sqrt{-23})$.

Let these splitting fields be L_f, L_g . Their intersection is Galois over $\mathbb{Q}$, so its Galois group is a common quotient of the two S_3 groups. The only possibilities are the trivial group, C_2 , and S_3 . The quadratic subfields differ, excluding C_2 and also excluding equality of the splitting fields. Therefore

$$L_f \cap L_g = \mathbb{Q}, \quad \text{Gal}(L_f L_g / \mathbb{Q}) \simeq S_3 \times S_3.$$

The group acts transitively on the nine ordered pairs of cubic roots.

No two distinct pairs have the same sum. Indeed, a coincidence $a_i + b_j = a_k + b_\ell$ gives

$$a_i - a_k = b_\ell - b_j \in L_f \cap L_g = \mathbb{Q}.$$

Two conjugates of an irreducible polynomial cannot differ by a nonzero rational translation: translating the minimal polynomial would leave it unchanged, which its next-to-leading coefficient rules out. Thus $i = k$ and $j = \ell$.

No two distinct pairs have the same product either. A coincidence gives $a_i/a_k = b_\ell/b_j \in \mathbb{Q}^\times$. If $a_i = ta_k$ with rational t , then the monic minimal polynomials $f(X)$ and $t^3 f(X/t)$ agree. Their constant terms give $t^3 = 1$, hence $t = 1$. Again the pairs coincide.

Consequently, the two transitive orbits each have nine distinct elements. The monic degree-nine resultants are the minimal polynomials of every associated sum or product, proving irreducibility. Each cubic has exactly one real root because its discriminant is negative. Complex conjugation fixes a sum or product in the nine-element orbit exactly when it fixes its unique ordered-pair label. That happens only for the pair of real roots. Thus each resultant has exactly one real root. Mathematica's real-root-first indexing therefore makes that root index one [2], proving (3)–(4). Combined with Theorem 7.1, this proves the two global minima.

9 The unrestricted sum problem has a finite reduction

The bounded catalog search is not the strongest possible additive algorithm. The key is to descend arbitrary summands into a fixed finite normal extension.

Theorem 9.1 (Degree-preserving additive descent). *Let $L/\mathbb{Q}$ be a finite Galois extension and $\alpha \in L$. If*

$$\alpha = \beta_1 + \cdots + \beta_r, \quad \beta_i \in \overline{\mathbb{Q}},$$

then there are $\gamma_i \in L \cap \mathbb{Q}(\beta_i)$ such that

$$\alpha = \gamma_1 + \cdots + \gamma_r, \quad \deg \gamma_i \leq \deg \beta_i.$$

Proof. Choose a finite Galois extension $E/\mathbb{Q}$ containing L and every β_i . Put $\Gamma = \text{Gal}(E/\mathbb{Q})$ and $H = \text{Gal}(E/L)$. Normality of $L/\mathbb{Q}$ makes H a normal subgroup of Γ . Define

$$T(v) = \frac{1}{|H|} \sum_{h \in H} h(v) = \frac{1}{[E:L]} \text{Tr}_{E/L}(v).$$

This is a $\mathbb{Q}$ -linear map $E \rightarrow L$ that fixes every element of L .

Normality also makes T equivariant under all of Γ : for $s \in \Gamma$, conjugation by s permutes H , giving $sT(v) = T(sv)$. If s fixes β_i , it therefore fixes $T(\beta_i)$. By Galois correspondence, $T(\beta_i)$ belongs to $\mathbb{Q}(\beta_i)$ as well as to L .

Set $\gamma_i = T(\beta_i)$. Then $\sum_i \gamma_i = T(\alpha) = \alpha$, and the field inclusion $\mathbb{Q}(\gamma_i) \subseteq \mathbb{Q}(\beta_i)$ proves the degree inequality. $\square$

Take L to be the normal closure of $\mathbb{Q}(\alpha)$ and set $G = \text{Gal}(L/\mathbb{Q})$. For a positive integer d , define the rational vector subspace

$$V_d(L) = \sum_{\substack{H \leq G \\ [G:H] \leq d}} L^H. \quad (9)$$

This is a sum of *embedded subfields as vector spaces*, not a field compositum.

Corollary 9.2 (Complete finite additive criterion). *For every $\alpha \in L$,*

$$D_+(\alpha) \leq d \iff \alpha \in V_d(L).$$

Consequently, $D_+(\alpha)$ and a minimizing representation can be computed by finitely many rational linear-algebra calculations after the normal closure is constructed.

Proof. A degree- d -bounded decomposition descends into L by Theorem 9.1. Each descended summand generates a subfield of degree at most d , hence belongs to a term of (9). Conversely, an expression for α in that vector-space sum is a sum of elements of subfields of degree at most d . Every such element has absolute degree at most d .

There are finitely many subgroups of finite G . It suffices to test $1 \leq d \leq \deg \alpha$, since α itself supplies a representation at the upper endpoint. $\square$

This resolves the unrestricted number of summands and the unrestricted coefficient heights for the additive problem. It is a finite algorithm, not a claim of practical efficiency for large splitting fields.

9.1 Constructing the fixed spaces exactly

Choose an algebraic-integer primitive element θ with $L = \mathbb{Q}(\theta)$ and let $N = [L : \mathbb{Q}]$. Use the power basis

$$1, \theta, \theta^2, \dots, \theta^{N-1}.$$

For $\sigma \in G$, let $M_\sigma \in \text{Mat}_N(\mathbb{Q})$ represent its action on coordinate columns. Then

$$L^H \longleftrightarrow \ker \begin{pmatrix} M_{h_1} - I \\ \vdots \\ M_{h_s} - I \end{pmatrix} \quad (10)$$

for a list of elements or generators of H . This is just the condition that every $h \in H$ fixes the element. Rational nullspace bases give columns spanning $V_d(L)$; solve

$$A_d c = \text{coord}(\alpha)$$

over $\mathbb{Q}$. Group coefficients belonging to each subfield basis and convert its contribution back to a single algebraic number. Rational linear combinations stay inside that subfield.

No transcendental logarithms, approximate relation recognition, or rational-point search is involved. A representation with at most N summands exists whenever the span condition holds: choose a basis of the span from the collected subfield vectors and absorb each rational scalar into its associated vector.

9.2 A certificate of nonmembership

When the linear system is inconsistent, one can exhibit a rational row vector w^T with

$$w^T A_d = 0, \quad w^T \text{coord}(\alpha) \neq 0.$$

This separating functional certifies that $\alpha \notin V_d(L)$. Together with the exhaustive subgroup list and exact automorphism matrices, it is a checkable lower-bound certificate. The package retains these functionals, rather than reporting an unexplained failure of `LinearSolve`.

9.3 What the implementation actually constructs

`GlobalSumDecompose` in the companion source performs the following finite steps.

1. It constructs the smallest common field of all target conjugates with the call `ToNumberField[roots, All]`. The `All` argument matters: the default is documented as not necessarily choosing the smallest common extension [7].
2. It chooses an algebraic-integer primitive generator and represents all its conjugates in the same field. The images of the primitive generator determine every automorphism. Their matrices and exact multiplication table are constructed by rational polynomial arithmetic modulo the generator's minimal polynomial.
3. It enumerates all subgroups, computes (10), and tests the increasing spaces $V_d(L)$. The first successful degree is globally minimal by Corollary 9.2.

The power-basis coordinates are extracted using `AlgebraicNumberPolynomial`. Its input comes from `ToNumberField` results [8]. It verifies that a coordinate conversion has not unexpectedly changed the generator.

A small example is

```

global = GlobalSumDecompose[Sqrt[2] + Sqrt[3]];
KeyTake[global, {"Components", "MaximumDegree",
  "ProvenMinimum", "FieldDegree", "GloballyOptimal"}]
(* Expected minimum: 2; splitting-field degree: 4. *)

```

For $\theta = \sqrt{2} + \sqrt{3}$, the minimal polynomial is $X^4 - 10X^2 + 1$. Its conjugates can be written as

$$\theta, \quad -\theta, \quad \theta^3 - 10\theta, \quad -\theta^3 + 10\theta.$$

These give four rational automorphism matrices. The degree-one fixed space does not contain θ ; the degree-two fixed spaces span the field. The independent Python checks verify this construction exactly.

The default resource caps are "MaxFieldDegree" $\rightarrow$ 72 and "MaxSubgroups" $\rightarrow$ 10000. Removing them with `Infinity` recovers the uncapped finite reference algorithm. A cap failure is a `Failure`, not a proof that no decomposition exists. The field-degree cap is checked *after* common-field construction; use an outer `TimeConstrained` to limit that initial step as well. The splitting field of either nonic in the question has degree 36, but constructing it is unnecessary once the simple cubic decomposition and lower bound have been found.

9.4 A real-summand variant

The supplied implementation allows complex algebraic summands, as in the definitions. If α and the initial summands are real, the trace proof still preserves reality: T commutes with complex conjugation. Thus the complete real-only criterion is obtained by restricting (9) to subgroups containing the complex-conjugation element of G . This is a mathematical variant, not an additional option silently enabled in the delivered implementation. Both motivating answers already use real components.

10 Why searching only the target field is not enough

A useful fast additive search is to supply several known subfields of an ambient field and solve a rational span problem:

```

SumOverSubfields[alpha, theta, {eta1, eta2, eta3}]

```

Here each η_i generates a specified subfield of $\mathbb{Q}(\theta)$. Its powers give a basis, their coordinate columns are concatenated, and exact linear algebra recovers one contribution from each field. Success gives a valid degree bound. Nonmembership concerns only the supplied subfields.

Replacing the normal closure in Theorem 9.1 by $\mathbb{Q}(\alpha)$ is not generally sound. The following explicit example shows that the restriction can change the optimum.

Example 10.1 (A degree-six number needing quartic summands outside its own field). Let $a_1, \dots, a_4$ be the roots of $F(X) = X^4 - X - 1$. Their six unordered pair sums are the roots of

$$P(Z) = Z^6 + 4Z^2 - 1,$$

as is also seen from

$$\text{Res}_X(F(X), F(Z - X)) = (Z^4 - 8Z - 16)(Z^6 + 4Z^2 - 1)^2.$$

The quartic is irreducible modulo two, its cubic resolvent $Y^3 + 4Y - 1$ has no rational root, and its discriminant is -283 . These facts give Galois group S_4 . The sextic is irreducible; it is also

checked exactly in the companion verification script. The action of S_4 on the six unordered pairs is faithful, so the normal closure of any pair sum has this same Galois group.

For any $\alpha = a_i + a_j$ with $i \neq j$, the quartic roots give $D_+(\alpha) \leq 4$. The order-four elements of S_4 and Theorem 7.2 exclude all inputs of degree at most three. Hence $D_+(\alpha) = 4$.

But $K = \mathbb{Q}(\alpha)$ has degree six. Degrees of its elements divide six, so its only possible element degrees below six are 1, 2, 3. There are no degree-four elements of K . A sum using only elements of K and having maximum degree below six would therefore contradict the preceding obstruction. The unrestricted optimum is four, while the optimum with every summand constrained to K is six.

This also explains why enumerating subfields only up to isomorphism or conjugacy can lose solutions: the algorithm needs the actual embedded fixed fields relevant to the target's coordinates.

10.1 More than two components can be essential

The number

$$\sqrt{2} + \sqrt{3} + \sqrt{5}$$

has degree eight and additive optimum two. It cannot be a sum of just two degree-two numbers, because their compositum has degree at most four. Likewise,

$$(1 + \sqrt{2})(1 + \sqrt{3})(1 + \sqrt{5})$$

has degree eight and product optimum two, but no two-factor degree-two representation. Both degree-eight claims are verified in `verify.py`. Thus an exhaustive pair search, even with unbounded coefficient height, is not a general solver for “several” components.

11 The product problem: what norms do and do not settle

For a fixed finite field L , let

$$B_d(L) = \langle F^\times : \mathbb{Q} \subseteq F \subseteq L, [F : \mathbb{Q}] \leq d \rangle \subseteq L^\times.$$

Membership $\alpha \in B_d(L)$ is exactly the product-decomposition problem *with all factors constrained to L* . This is a multiplicative subgroup problem, not a rational vector-space problem. A basis of the subfields under addition does not generate their multiplicative groups.

Proposition 11.1 (A necessary norm-descent condition). *If $L/\mathbb{Q}$ is Galois, $\alpha \in L^\times$, and α is a product of algebraic numbers of degrees at most d , then $\alpha^M \in B_d(L)$ for some positive integer M .*

Proof. Choose a common finite Galois extension $E/\mathbb{Q}$ containing all factors and L . With $H = \text{Gal}(E/L)$, put $M = |H|$ and $\delta_i = \prod_{h \in H} h(\beta_i)$. Normality of H shows, as in the trace proof, that every automorphism fixing β_i fixes δ_i . Thus $\delta_i \in L \cap \mathbb{Q}(\beta_i)$ and $\deg \delta_i \leq d$. Taking the norm of $\alpha = \prod_i \beta_i$ gives $\alpha^M = \prod_i \delta_i$. $\square$

The conclusion concerns the *saturation* of $B_d(L)$, not $B_d(L)$ itself. The distinction cannot be dropped: $\sqrt{2}$ has a rational square, but it is not a product of rational numbers. Thus even at $d = 1$, a power-membership certificate is not a decomposition certificate. Taking M th roots can increase absolute degree and introduce root-of-unity ambiguities.

A related refinement appears in Samuels's norm-based reduction for metric Mahler measure [11]. Put $F_i = L \cap \mathbb{Q}(\beta_i)$ and $k_i = [\mathbb{Q}(\beta_i) : F_i]$. Taking a k_i th root η_i of $N_{\mathbb{Q}(\beta_i)/F_i}(\beta_i)$ gives

$$\eta_i^{k_i} \in F_i, \quad \deg \eta_i \leq k_i[F_i : \mathbb{Q}] = \deg \beta_i, \quad \alpha = \zeta \prod_i \eta_i$$

for some root of unity ζ . This usefully restricts replacement factors to radicals of the normal closure. It does not solve the present optimization: the extra root of unity is not free under an absolute-degree cost, and replacing radical factors by elements of L requires further justification. Consequently, this article does *not* claim that `GlobalSumDecompose` can be converted into a complete product optimizer by replacing traces with norms, or by taking numerical logarithms and applying `LinearSolve`. Exact multiplicative relations, valuations, units, and roots of unity require separate treatment. A number-field implementation may use such arithmetic to improve its searches, but a finite ideal-support restriction or a saturation step needs its own correctness proof.

The delivered product routines provide two rigorous forms of answer: a globally optimal decomposition when an attained upper bound meets a proved lower bound, and an exhaustive result within explicit degree, height, and component-count bounds otherwise. For the two examples in the question, the first form applies.

12 Implementation contract and practical workflow

The principal entry points in `RootDecomposition.wl` are summarized below. All polynomial and algebraic inputs are exact; floating-point input is rejected rather than rationalized silently.

Function	Guarantee or purpose
<code>AlgebraicDegree</code>	Absolute degree from a rational minimal polynomial.
<code>DegreeLowerBound</code>	Lower bound valid for any finite number of components.
<code>ComposedPolynomial</code>	All pairwise sums or products, with multiplicities.
<code>RootPairFromPolynomials</code>	Exact branch selection and a checked decomposition.
<code>PairCoefficientEquations</code>	Inverse resultant equations; rational coefficient solutions are required.
<code>BoundedPairDecompose</code>	Complete finite search over the stated two-component catalog.
<code>BoundedDecompose</code>	Complete finite search with an additional component-count bound.
<code>SumOverSubfields</code>	Exact additive span test for specified embedded fields.
<code>GlobalSumDecompose</code>	Globally minimal additive result if construction completes within its resource caps.

A sensible order of attack is to compute the true minimal polynomial and an elementary lower bound, try a small catalog or a structurally suggested inverse-resultant ansatz, and certify any result exactly. Stop immediately when the maximum component degree reaches the proved bound. For sums without a matching certificate, use supplied subfields first and the complete normal-closure algorithm only when the field size is manageable. For products without a matching bound, report the best certified upper bound and the remaining gap instead of presenting bounded non-discovery as impossibility.

If a computation is time-limited, preserve the distinction between a resource limit and a mathematical negative answer:

```
TimeConstrained[
  GlobalSumDecompose[alpha],
  120,
  Failure["Timeout", <|"MessageTemplate" ->
    "The time limit was reached; no impossibility is proved."|>]
]
```


For a successful complete additive call, the full association can be large because it retains matrices, subgroups, and lower-bound witnesses. Use `KeyTake` to display only the desired summary.

12.1 Exiting nested searches reliably

The search routines use a locally tagged `Catch/Throw` pair to return a witness or propagate a failure through every nested loop. This is deliberate: `Return` inside `Do` can return from that loop without returning from the enclosing search function [9]. A found decomposition must not be discarded merely because another loop resumes. The supplied regression specifications include nested resource-limit and invalid-subfield failures.

13 Validation, reproducibility, and limitations

Executed checks. The included `verify.py` completed 30 checks using exact SymPy arithmetic. They cover both given resultants, irreducibility of the cubics and nonics, exactly one real root for each nonic, the discriminants, the sparse-ansatz coefficient formulas, both companion-matrix identities, nonmonic normalization, zero and repeated-factor cases, bounded catalog discovery, the biquadratic fixed-space construction, the quartic/sextic counterexample, and the degree-eight three-component examples. The generated `verification_results.json` records the Python and SymPy versions, every check, and the independently discovered polynomial pairs.

The numerical approximations in this article are supplementary descriptions. The proofs and acceptance criteria use exact polynomials and exact algebraic equality, not decimal tolerances.

Wolfram execution status. A Wolfram connector was available for discovery, but both the context request and the attempted kernel evaluation returned an HTTP 404 error from its MCP endpoint. No Mathematica kernel output was obtained. The Wolfram Language source was written against the cited official interfaces and reviewed, but it is not represented here as kernel-tested. In particular, the full normal-closure implementation has not been executed in a Wolfram kernel during preparation of this article. The independent tests check its mathematical construction on a small field, not every Wolfram implementation detail.

Run the supplied regression specifications locally with

```
TestReport["Tests.wlt"]
```

The tests include the two identities, bounded automatic discovery, exact input rejection, rational coefficient enforcement, field-span behavior, a complete biquadratic additive calculation, a cyclic-quartic minimum of four, and resource-limit handling. Some algebraic field computations can be slow; test expectations are not timing promises.

Scope of completeness. The unrestricted additive algorithm is justified by Theorem 9.1 and Corollary 9.2; practical caps can interrupt it. The multiplicative implementation is a certified bounded search combined with global lower bounds, not a claimed terminating unrestricted optimizer. This is a limitation of the delivered algorithm, not an assertion that the unrestricted product problem is undecidable or that no stronger algorithm can exist.

14 Conclusion

The two numbers in the question have the desired cubic product and cubic sum representations, and degree three is genuinely optimal even when arbitrarily many algebraic components are allowed.

A reliable implementation separates three tasks: *discover* candidate rational polynomials, *select and certify* the correct algebraic branches, and *prove optimality* with an independent lower bound. Resultants solve the forward algebra and provide exact inverse equations; explicit bounds make exhaustive discovery honest and reproducible.

For addition, normalized trace supplies a complete finite reduction to fixed subfields of the target’s normal closure. For multiplication, norm descent leaves a power and radical obstruction, so the same linear-algebra argument must not be reused without further mathematics. Keeping that distinction explicit is what allows the software to return useful decompositions without overclaiming what a finite search has proved.

A Archive contents and build instructions

The archive is self-contained for compiling the article, apart from standard LaTeX packages. It contains no external fonts or downloaded research papers.

File	Contents
<code>article.tex</code> , <code>article.pdf</code>	Source and compiled article.
<code>RootDecomposition.wl</code>	Full Wolfram Language reference package.
<code>Examples.wl</code>	Direct and bounded-discovery examples.
<code>Tests.wlt</code>	Mathematica regression specifications.
<code>verify.py</code>	Independently executable SymPy verification.
<code>verification_results.json</code>	Results of the executed independent checks.
<code>README.md</code>	Scope, usage, test status, and build instructions.

Compile from the archive directory with two passes:

```
pdflatex -interaction=nonstopmode -halt-on-error article.tex
pdflatex -interaction=nonstopmode -halt-on-error article.tex
python verify.py
```

The bibliography is embedded below, so BibTeX and external source files are not needed.

References

- [1] V. Reshetnikov, “Factor a polynomial Root into Roots of smallest possible degree,” Mathematica Stack Exchange, question 105933 (2016), with answers by yarchik and Daniel Lichtblau (2019). <https://mathematica.stackexchange.com/questions/105933/>.
- [2] Wolfram Research, *Root*, Wolfram Language Documentation. <https://reference.wolfram.com/language/ref/Root.html>.
- [3] Wolfram Research, *RootReduce*, Wolfram Language Documentation. <https://reference.wolfram.com/language/ref/RootReduce.html>.
- [4] Wolfram Research, *Resultant*, Wolfram Language Documentation. <https://reference.wolfram.com/language/ref/Resultant.html>.
- [5] Wolfram Research, *MinimalPolynomial*, Wolfram Language Documentation. <https://reference.wolfram.com/language/ref/MinimalPolynomial.html>.
- [6] Wolfram Research, *IrreduciblePolynomialQ*, Wolfram Language Documentation. <https://reference.wolfram.com/language/ref/IrreduciblePolynomialQ.html>.

- [7] Wolfram Research, *ToNumberField*, Wolfram Language Documentation. <https://reference.wolfram.com/language/ref/ToNumberField.html>.
- [8] Wolfram Research, *AlgebraicNumberPolynomial*, Wolfram Language Documentation. <https://reference.wolfram.com/language/ref/AlgebraicNumberPolynomial.html>.
- [9] Wolfram Research, *Return*, Wolfram Language Documentation, especially “Possible Issues.” <https://reference.wolfram.com/language/ref/Return.html>.
- [10] J. S. Milne, *Fields and Galois Theory*, version 5.10, September 2022. In particular, Chapter 3, Theorem 3.17, and Chapter 5 on norms and traces. <https://www.jmilne.org/math/CourseNotes/FT.pdf>.
- [11] C. L. Samuels, *The infimum in the metric Mahler measure*, arXiv:1408.4165, 2014 version. See Theorem 2.1, Lemma 2.2, and the degree estimate in its proof. <https://arxiv.org/abs/1408.4165>.

All online references were consulted on September 6, 2026.